Lakeflow Connect & Data Sources

- ✦ Lakeflow Connect: All connector types (SaaS, Database, Query-based)
- ✦ JDBC/ODBC ingestion from relational databases
- ✦ REST API ingestion with pagination
- ✦ Semi-structured data: JSON, structs, arrays
- ✦ Secrets management (Workspace & Unity Catalog)
- ✦ Complete Domain 2 decision framework (updated)
- ✦ Practice Questions & Mock Exam (15 Qs)

Table of Contents

1. Lakeflow Connect Overview
2. Connector Types (REVISED)
3. SaaS Connectors
4. Database Connectors & CDC
5. Query-Based Connectors (NEW)
6. JDBC/ODBC Ingestion
7. Secrets Management
8. REST API Ingestion
9. Semi-Structured Data
10. Domain 2 Decision Framework
11. Practice Questions (10 Qs)
12. Answer Key
BONUS: Advanced Patterns
BONUS: Mock Exam (15 Qs)
Cheat Sheet & Self-Assessment

1	Lakeflow Connect Overview	Managed data ingestion

Lakeflow Connect is Databricks' managed connector service for ingesting data from SaaS applications, databases, and enterprise systems into the Lakehouse. It eliminates custom ETL code for common data sources, handling authentication, schema mapping, and incremental sync automatically.

1.1 Service Models

Lakeflow Connect offers a flexible architecture with two main approaches:

■ Service Models

```
1. Fully-managed service:
   - Out-of-the-box connectors (no code)
   - Simple UI configuration
   - Built-in authentication, CDC, schema evolution

2. Custom pipeline (for advanced customization):
   - Lakeflow pipelines or Structured Streaming
   - Mix managed + custom logic
```

1.2 Why Lakeflow Connect vs Custom Code?

Consideration	Lakeflow Connect	Custom Code
Development time	Minutes	Hours to days
Maintenance	Databricks managed	Engineer maintained
API changes	Auto-handled	Manual updates
Incremental sync	Built-in	Custom logic needed

■ **Exam Focus** Lakeflow Connect is the recommended path for most scenarios.

2 Connector Types (REVISED)

Complete taxonomy 2026

Key Update: Databricks now organizes all connectors by *source type*, not by 'Standard' vs 'Managed'. There are 6 connector types:

2.1 Six Connector Types

Type	Examples	Description
SaaS Connectors	Salesforce, Workday, HubSpot	Databricks-built; UI/API; OAuth
Database (CDC)	MySQL, PostgreSQL, SQL Server	Change Data Capture; gateway-based
Query-Based (NEW)	Oracle, Teradata, SQL Server	Direct query; no CDC; GA May 2026
File Source	Google Drive, SharePoint	Unstructured/structured files
Streaming	RabbitMQ, Kafka	Continuous message consumption
Community	Various open-source	Community-maintained

2.2 All Build on Lakeflow Pipelines + Serverless

Every managed connector runs as a Lakeflow pipeline on serverless compute, writes to streaming tables, and auto-creates jobs for orchestration. The deployment is unified across all types.

■■ **Warning No more 'Standard' and 'Managed' labels.** All are managed. Categorization is by source type (SaaS, Database, etc.)

3	SaaS Connectors	Salesforce, Workday, HubSpot...

3.1 UI-Driven Setup

■ **SaaS Setup**

```
Step 1: Ingestion → Lakeflow Connect
Step 2: Select connector (Salesforce, Workday, etc.)
Step 3: Authenticate (OAuth or API key)
Step 4: Select objects (Accounts, Opportunities, etc.)
Step 5: Configure target catalog.schema
Step 6: Set schedule (hourly, daily)
Step 7: Databricks auto-creates DLT pipeline + job
Step 8: Monitor in Lakeflow Jobs UI
```

3.2 Sync Modes

Mode	Use Case	Behavior
Incremental	Operational daily sync	Only changed records via timestamps
Full refresh	Corrections or schema changes	Re-sync entire dataset
CDC	Near-real-time	Insert/update/delete events

■ **Tip** For SaaS, incremental sync is automatic and efficient.

4	Database Connectors	CDC for MySQL, PostgreSQL, SQL Server

4.1 CDC Architecture

Database connectors use **Change Data Capture** to track source updates. Requires an ingestion gateway running in your workspace to capture change events continuously.

Component	Role
Connection	Stores DB credentials in Unity Catalog
Ingestion Gateway	Captures CDC events (continuous job)
Staging Storage	Temporary CDC log storage
Pipeline	Processes CDC → writes to streaming table

4.2 Programmatic Connection Creation

■ CLI Connection Creation

```
databricks connections create \
  --type MYSQL \
  --name my_mysql_conn \
  --json '{"host":"mysql.example.com", "port":3306, ...}'
```

■■ **Warning** Some SaaS connectors (Jira, Slack, Google Ads, HubSpot) require browser-based OAuth and cannot be created programmatically.

5 Query-Based Connectors NEW (May 2026 GA)

Query-based connectors query the source database directly on a schedule **without CDC** or a gateway. Simpler setup, still incremental, ideal for periodic batch loads.

5.1 Query-Based vs CDC Comparison

Aspect	Query-Based	Database (CDC)
Gateway	Not required	Required
Setup	Simpler	More complex
Incremental method	Cursor column (timestamp)	Change events
Latency	Scheduled batches	Near-real-time
When to use	Periodic batch, no CDC need	Continuous/event-driven

5.2 Example Configuration

■ **Query-Based Oracle Config**

```
{
  "connector_type": "oracle_query_based",
  "source_config": {
    "host": "oracle.company.com",
    "cursor_column": "LAST_MODIFIED_DATE"
  },
  "objects": [{"source_table": "ORDERS"}]
}
```

■ **Exam Focus** Query-based connectors reduce operational complexity for batch ingestion scenarios.

6.1 When to Use JDBC

Use JDBC if: No connector exists or you need extreme customization. Otherwise, use Lakeflow Connect (SaaS, Query-based, or CDC).

6.2 Basic Read

■ **Basic JDBC**

```
df = (spark.read.format("jdbc")
  .option("url", "jdbc:postgresql://host:5432/mydb")
  .option("dbtable", "public.orders")
  .option("user", dbutils.secrets.get("scope", "pg_user"))
  .option("password", dbutils.secrets.get("scope", "pg_pass"))
  .load()
)
```

6.3 Parallel Read (CRITICAL)

All 4 options must be set together for parallelism:

■ **JDBC Parallel Read**

```
.option("partitionColumn", "order_id")
.option("lowerBound", "1")
.option("upperBound", "10000000")
.option("numPartitions", "20")
```

Option	Purpose
partitionColumn	Numeric or date column
lowerBound	Range start
upperBound	Range end
numPartitions	Parallel tasks

■ **Exam Focus Setting just partitionColumn alone has no effect.** All 4 must be set together.

7.1 Workspace Secret Scopes (Original)

■ **Workspace Secret Scope**

```
# Create scope
databricks secrets create-scope jdbc

# Add secrets
databricks secrets put-secret jdbc username
databricks secrets put-secret jdbc password

# Access in notebook
user = dbutils.secrets.get("jdbc", "username")
pwd = dbutils.secrets.get("jdbc", "password")
# print(pwd) → [REDACTED]
```

7.2 Unity Catalog Secrets (NEW — GA August 2026)

■ **Unity Catalog Secret**

```
# Create UC secret (3-level namespace)
dbutils.secrets.create_secret(
    scope="main.secure",  # catalog.schema
    key="db_password",
    string_value="MyPass123"
)

# Access
pwd = dbutils.secrets.get("main.secure", "db_password")
```

Feature	Workspace	Unity Catalog
Multi-workspace	No	Yes (metastore-level)
Namespace	scope.key	catalog.schema.secret
Governance	Basic ACLs	Full UC privileges
Audit	Basic	Full system.access.audit

■ **Tip Best practice:** Workspace scopes for prototyping; UC secrets for production.

8.1 Simple Pattern

■ **REST → Delta**

```
import requests
from pyspark.sql.functions import current_timestamp

# Fetch from API
resp = requests.get("https://api.example.com/events",
    headers={"Authorization": f"Bearer {dbutils.secrets.get('scope','token')}"})

# Create DataFrame
data = resp.json()["events"]
df = spark.createDataFrame(data)
df = df.withColumn("_ingest_ts", current_timestamp())

# Write to Delta
df.write.format("delta").mode("append").saveAsTable("bronze.api_events")
```

8.2 Pagination

■ **Paginated API**

```
all_records = []
page = 1
while True:
    resp = requests.get(f"https://api.example.com/data?page={page}")
    batch = resp.json().get("data", [])
    if not batch: break
    all_records.extend(batch)
    page += 1

df = spark.createDataFrame(all_records)
df.write.format("delta").mode("overwrite").saveAsTable("bronze.api_data")
```

9.1 JSON Functions Quick Reference

Function	Input	Output	Use
col("a.b")	StructType	scalar	Nested struct access
raw:field	STRING (JSON)	STRING	SQL colon operator
from_json()	STRING	StructType	Parse full JSON
get_json_object()	STRING	STRING	Extract one field
explode()	ArrayType	Rows	Flatten array

9.2 Colon Operator (SQL Only)

■ Colon Operator

```
SELECT
  raw:user.id,
  raw:events[0].type,
  raw:metadata.city
FROM bronze.raw_events
```

9.3 Exploding Arrays

■ Flatten Arrays

```
# Input: {"order_id":"O1", "items":[{"sku":"A"},{"sku":"B"}]}
df_flat = df.select(
    col("order_id"),
    explode(col("items")).alias("item")
)
df_flat.select("order_id", "item.sku")
```

10 Domain 2 Decision Framework

All tools + logic

Scenario	Tool	Key Reason
Batch from cloud storage (S3/ADLS)	COPY INTO	Idempotent, tracks files
Continuous streaming files	Auto Loader	Checkpointing, schema evolution
Salesforce, Workday, HubSpot	Lakeflow SaaS	Managed, no-code
MySQL, PostgreSQL, real-time	Database (CDC)	Partner connector, real-time
Oracle, MySQL, periodic batch (NEW)	Query-Based	No gateway, simpler
Custom JDBC driver	JDBC	Universal, parallelizable
Custom REST API	Python requests	Flexible pagination
Nested JSON / arrays	from_json/explode	Databricks native

■ **Exam Focus Exam logic:** SaaS → Lakeflow. DB → Query-based (batch) or CDC (continuous). Files → COPY/Auto Loader. Custom → REST/JDBC.

1. Company syncs Salesforce daily, no custom code needed. Which tool?

A) JDBC
B) Lakeflow SaaS Connector
C) Auto Loader
D) REST API

2. PostgreSQL 100M rows via JDBC, only 1 task used, too slow. Solution?

A) Increase fetchsize
B) Use parallelReads option
C) Set ALL 4: partitionColumn + bounds + numPartitions
D) Use Auto Loader

3. JSON STRING column 'raw', extract user.city in SQL?

A) raw['user']['city']
B) raw:user.city
C) get_json_object(raw)
D) json_extract(raw)

4. Query-Based Connectors (May 2026 GA) — what's key advantage over CDC?

A) Faster throughput
B) No gateway needed; simpler setup
C) Better for real-time
D) Cheaper

5. Store credentials — workspace scope or UC secret?

A) Always workspace
B) UC secrets for production & governance
C) Identical
D) Workspace is free

6. ARRAY> — create 1 row per item?

A) flatten()
B) explode()
C) unnest()
D) lateral_view()

7. Best tool for HubSpot ingestion?

A) JDBC
B) Lakeflow SaaS
C) Auto Loader
D) Custom REST

8. from_json() vs get_json_object() key difference?

A) Both return STRING
B) from_json→StructType, get_json→STRING
C) Opposite
D) Same, different name

9. REST API paginated results (list) → Delta next step?

A) COPY INTO
B) spark.createDataFrame(list) + write
C) Pandas
D) DBFS put

10. When to choose Query-Based over CDC?

A) Real-time requirement

B) Periodic batch, no CDC need

C) High volume

D) Event-driven

12 **Answer Key** Solutions

#	Answer	Explanation
1	**B**	Lakeflow SaaS Connectors = UI-only, no-code for Salesforce, Workday, HubSpot.
2	**C**	All 4 JDBC parallel options must be set together.
3	**B**	Colon (:) is Databricks' native SQL JSON operator.
4	**B**	Query-Based = no gateway, simpler setup, GA May 2026.
5	**B**	UC secrets = 3-level namespace + full UC governance (production standard).
6	**B**	explode() flattens arrays to rows.
7	**B**	HubSpot = SaaS = Lakeflow SaaS Connector.
8	**B**	from_json → StructType (typed), get_json_object → STRING.
9	**B**	spark.createDataFrame(list) converts Python list to DataFrame.
10	**B**	Query-Based ideal for periodic batch without CDC complexity.

A. schema_of_json() — Infer Unknown JSON Schema

Infer Schema

```
from pyspark.sql.functions import schema_of_json, from_json, lit

sample = '{"user_id":"u1","score":42}'
schema = schema_of_json(lit(sample))

df_parsed = df.withColumn("data", from_json(col("raw"), schema))
df_parsed.select("data.user_id", "data.score")
```

B. Parse & Flatten Arrays of Structs

Arrays of Structs

```
# {"order_id":"O1", "items":[{"sku":"A","qty":2}]}
from pyspark.sql.types import *

item_schema = ArrayType(StructType([
    StructField("sku", StringType()),
    StructField("qty", IntegerType())
]))

order_schema = StructType([
    StructField("order_id", StringType()),
    StructField("items", item_schema)
])

df_parsed = df.withColumn("order", from_json(col("raw"), order_schema))
df_flat = df_parsed.select(
    col("order.order_id"),
    explode(col("order.items")).alias("item")
)
```

C. JDBC dbtable vs query

dbtable vs query

```
# Full table (supports parallelization)
.option("dbtable", "orders")

# Custom query (no parallelization)
.option("query", "SELECT * FROM orders WHERE status='active'")

# Workaround: wrap as subquery
.option("dbtable", "(SELECT * FROM orders WHERE status='active') t")
.option("partitionColumn", "order_id")  # now supported
```

MOCK **Mock Exam** 15 Advanced Questions

1. 50GB daily from Oracle, stable, no real-time. Simplest tool?

A) CDC
B) Query-Based Connector
C) JDBC
D) DLT

2. Auto Loader with file notification vs directory listing — key advantage?

A) Cheaper
B) Event-driven; no full scan
C) Faster parsing
D) Schema inference

3. COPY INTO re-ingests loaded files with?

A) .option('force','true')
B) .option('mode','overwrite')
C) TRUNCATE then COPY
D) New connector

4. REST API 10K records/page, 50 pages → DataFrame?

A) RDD.toDF()
B) spark.createDataFrame(list)
C) parallelize()
D) fromJSON()

5. Salesforce connector from workspace A → workspace B?

A) Git Folders only
B) Declarative Bundle
C) Workspace-local only
D) API clone

6. SaaS connector default auth?

A) Service principal
B) OAuth 2.0 / API key (UC connection)
C) SQL login
D) PAT

7. dbutils.secrets.get prints actual secret?

A) Yes
B) [REDACTED] (auto-redacted)
C) scope:key
D) Error thrown

8. JDBC: partitionColumn + 3 others → parallel DB queries?

A) 1
B) = numPartitions value
C) Depends on cluster
D) All at once

9. SQL extract nested city from JSON string?

A) raw.user.address.city
B) raw:user.address.city
C) get_json_object(raw,path)
D) raw['user']['address']

10. Query-based vs CDC sync tracking difference?

A) Query: cursor column; CDC: events
B) Identical
C) Query needs gateway
D) CDC always faster

11. Lakeflow SaaS connectors use which model?

A) Pull from source on schedule
B) Push from source (CDC)
C) JDBC polling
D) Streaming API

12. Unity Catalog secret namespace format?

A) scope.key
B) catalog.schema.secret
C) workspace.scope.key
D) account.vault.secret

13. Access level needed to reference UC secret in pipeline?

A) READ SECRET
B) REFERENCE SECRET
C) USE SCHEMA
D) MANAGE CONNECTION

14. explode(array_col) with 1M rows × 5-item arrays → result?

A) 1M rows
B) ~5M rows
C) 1M/5 rows
D) Undefined

15. Best practice: workspace scope or UC secret for production?

A) Workspace (simple)
B) UC secrets (governance + audit)
C) Identical
D) Workspace (free)

CS **Cheat Sheet** Quick Reference

Connector Selector

Scenario	Tool
SaaS (Salesforce, Workday, etc.)	Lakeflow SaaS
Relational DB, periodic batch	Query-Based Connector
Relational DB, real-time	Database (CDC)
S3/ADLS batch	COPY INTO
Files arrive continuously	Auto Loader
Custom JDBC driver	spark.read.format('jdbc')
Custom REST API	Python requests + createDataFrame
JSON nested/arrays	from_json() / explode()

JSON & Secrets Syntax

Tool	Syntax
Workspace secret	dbutils.secrets.get("scope", "key")
UC secret	dbutils.secrets.get("catalog.schema", "key")
Colon operator	raw:user.id (SQL only)
from_json	from_json(col("raw"), schema)
get_json_object	get_json_object(col("raw"), "$.path")
explode	explode(col("array_col"))
JDBC parallel	partitionColumn + lowerBound + upperBound + numPartitions (all 4)

SA **Self-Assessment** Week 5 Mastery

✓ 6 connector types (SaaS, CDC, Query-based, File, Streaming, Community)

✓ Query-Based Connectors (May 2026 GA) — simpler than CDC

✓ When to use Lakeflow vs JDBC vs REST vs Auto Loader

✓ JDBC parallel read: ALL 4 options required

✓ dbutils.secrets.get() — never hardcode credentials

✓ Workspace scopes vs UC secrets (namespace & governance)

✓ Colon operator for JSON extraction in SQL

✓ from_json (→ StructType) vs get_json_object (→ STRING)

✓ explode() to flatten arrays

✓ REST API pagination + spark.createDataFrame

✓ Complete Domain 2 decision framework

✓ Incremental sync: cursor columns, watermarks, CDC events

✓ Phase 2 (Weeks 3-5) complete → Domain 2 (21%) secured

DEEP **Troubleshooting & Common Issues** Production scenarios

A.1 JDBC Connection Issues

Problem: 'Unable to find JDBC driver'

Solution: Driver class and URL format must match exactly. Ensure driver JAR is available on cluster.

A.2 Secret Scope Access Denied

Problem: dbutils.secrets.get() throws 'Access Denied'

Solution: Verify ACL permissions. For workspace scopes, grant READ to principal. For UC secrets, grant READ SECRET privilege.

A.3 Auto Loader Checkpointing Issues

■ **Auto Loader Checkpoint Reset**

```
# Issue: Schema mismatch or corrupt checkpoint
# Solution: Reset checkpoint location (loses state)
dbutils.fs.rm('path/to/checkpoint', True)

# Restart Auto Loader with new checkpoint path
df = spark.readStream.format('cloudFiles')
  .option('cloudFiles.format', 'parquet')
  .option('checkpointLocation', 'new/checkpoint/path')
  .load(path)
```

A.4 Lakeflow Connect Connector Failure

Problem: Connector failed, missed data

Diagnosis: Check Lakeflow Jobs run history. Look for error logs in cluster logs.

Recovery: Connector auto-retries with exponential backoff. If manual intervention needed, manually update the cursor column to resume from last valid state.

B. Performance Optimization

B.1 JDBC Read Optimization

Optimization	Effect	Trade-off
Increase numPartitions	More parallel tasks	More DB connections
Adjust fetchsize	Smaller batches faster	More network round-trips
Use query (not dbtable)	Filter at source	No parallelization
Enable compression	Smaller network payload	CPU overhead on client

B.2 Auto Loader Tuning

■ **Auto Loader Tuning**

```
# Optimize for high file volume
df = spark.readStream.format('cloudFiles')
  .option('cloudFiles.format', 'parquet')
  .option('cloudFiles.useNotifications', 'true')  # Event-driven vs polling
  .option('cloudFiles.schemaEvolutionMode', 'addNewColumns')
  .option('cloudFiles.maxFileAge', '6h')  # Don't re-process old files
  .option('maxFilesPerTrigger', 10000)  # Limit batch size
  .load(path)
```

APP

JDBC Driver Reference

Connection strings & drivers

Database Connection Details

Database	JDBC URL Pattern	Driver Class
PostgreSQL	jdbc:postgresql://host:5432/db	org.postgresql.Driver
MySQL	jdbc:mysql://host:3306/db	com.mysql.cj.jdbc.Driver
SQL Server	jdbc:sqlserver://host:1433;databaseName=db	com.microsoft.sqlserver.jdbc.SQLServerDriver
Oracle	jdbc:oracle:thin:@host:1521:SID	oracle.jdbc.driver.OracleDriver
Redshift	jdbc:redshift://host:5439/db	com.amazon.redshift.jdbc.Driver

FLOW **Lakeflow Connect Lifecycle** Setup to production

Complete Setup Workflow

1. **Create Connection** (UI or API)
   - Credentials stored in UC connection object
2. **Configure Ingestion Pipeline**
   - Select source objects
   - Map to target catalog.schema
   - Choose sync mode (incremental/full/CDC)
3. **Set Schedule**
   - Hourly, daily, custom cron
   - Databricks auto-creates Lakeflow Jobs
4. **Monitor & Maintain**
   - Check Lakeflow Jobs run history
   - Monitor data freshness
   - Handle auth failures (token expiry, etc.)

Production Considerations

Aspect	Best Practice
Credentials	Use UC connection (Unity Catalog managed)
Monitoring	Set up alerts on job failures; check audit logs
Cost tracking	Query system.billing.usage for connector costs
Schema changes	Use schema evolution; handle new columns gracefully
Backups	Ensure bronze layer has time-travel (Delta Lake ACID)
Documentation	Version connector configs in Git via Declarative Bundles

EXAM **Exam Strategy** Domain 2 tactics (21%)

Domain 2 Distribution (~9 Questions out of 45)

~3 questions: Connector selection (SaaS vs CDC vs Query-based vs JDBC vs REST)

~2 questions: JDBC parallel read configuration

~2 questions: Semi-structured data (JSON, explode, colon operator)

~2 questions: Auto Loader vs COPY INTO, schema evolution, secrets

Question Patterns to Watch

Pattern 1 — Connector Selection: Scenario describes a source (Salesforce, Oracle, S3, REST API). Pick the right tool. Use the decision framework.

Pattern 2 — JDBC Tuning: 'Read from MySQL 100M rows; slow.' Answer: parallelization (ALL 4 options).

Pattern 3 — JSON Handling: Parse nested JSON or flatten arrays. Know from_json(), explode(), colon op.

Pattern 4 — Secrets & Security: Credentials stored where? dbutils.secrets.get() for workspace scopes; UC connection for Lakeflow.

Pattern 5 — Incremental Sync: How does connector track changes? CDC, cursor column, watermark, or timestamp?

Test Day Checklist

✓ Read the full scenario before picking answer (multi-part context)

✓ Identify the SOURCE: SaaS, DB, REST, Files?

✓ Identify the REQUIREMENT: batch, continuous, real-time?

✓ Use decision framework to narrow to 1-2 tools

✓ Check for gotchas: 'no code' → Lakeflow, 'custom JDBC' → JDBC, 'raw JSON' → colon op or from_json

✓ Revisit JDBC parallel: all 4 options or no parallelization

✓ Double-check: dbutils.secrets vs UC secrets vs plaintext

SUMMARY **Week 5 Summary** Phase 2 Complete

What You've Mastered

- All 6 Lakeflow Connect connector types (SaaS, CDC, Query-based, File, Streaming, Community)
- Query-Based Connectors (May 2026 GA) — no CDC, no gateway
- SaaS connectors: UI-driven, OAuth, incremental sync automatic
- Database connectors: CDC-based, ingestion gateway, real-time
- JDBC: parallel reads with 4-option requirement
- Secrets: workspace scopes vs Unity Catalog (governance & namespace)
- REST APIs: pagination pattern, requests library
- JSON handling: from_json, get_json_object, colon operator, explode
- Decision framework: SaaS → Lakeflow, DB → Query-based/CDC, Files → COPY/Auto Loader
- Domain 2 coverage: 21% of exam = ~9 questions (LOCKED IN)

Confidence Level After Week 5

You should feel confident discussing and selecting the right ingestion tool for any scenario. Week 5 + Weeks 3-4 = complete mastery of **Domain 2 (21% of exam)**.

Next: Phase 3 (Weeks 6-8) — Data Transformation & Modeling

Domain 3 (22% of exam). Topics: Bronze → Silver → Gold patterns, joins (broadcast hint), aggregations, materialized views, data quality checks, Spark tuning parameters.

Cumulative Progress: Domains 1 + 2 complete (6% + 21% = 27% of exam). Moving into transformation territory—the heavy lifting section.

CASES	Real-World Case Studies	Production lessons learned

Case 1: E-Commerce Retailer — SaaS + JDBC Hybrid

Requirement: Ingest Salesforce customer data + PostgreSQL order history + S3 clickstream logs, unified daily report.

Solution:

• Salesforce → Lakeflow SaaS Connector (daily incremental)
• PostgreSQL → Query-Based Connector (parallel batch, cursor on order_date)
• S3 clickstream → Auto Loader (directory listing, hourly batch)
• All land in bronze layer (UC-governed)
• Silver layer: join & deduplicate
• Gold layer: daily report table for BI team

Case 2: Financial Services — Real-Time CDC from Oracle

Requirement: Ingest Oracle GL transactions in near-real-time (sub-minute latency). Audit trail must be immutable.

Solution:

• Oracle Database Connector (CDC mode) with ingestion gateway
• Streaming table consumes CDC events (insert/update/delete tracked)
• Delta Lake provides ACID + time-travel for regulatory audit
• Lakeflow Job runs every 30 sec to process CDC backlog
• Fail-safe: cursor stored in UC, auto-resume from last checkpoint

Case 3: SaaS Analytics Platform — Custom REST API + Pagination

Requirement: Ingest customer usage events from custom REST API (100K events/day, paginated). No Lakeflow Connect connector exists.

Solution:

• Python notebook with requests library + pagination loop
• Fetch all pages, concatenate into single DataFrame
• Add _ingest_ts column (Databricks timestamp)
• Write to bronze as delta (append mode for idempotency)
• Orchestrate via Lakeflow Jobs (daily job at 2 AM)
• Error handling: store checkpoint with last page number, resume next run

QA **Data Quality Patterns** Best practices

Q.1 Validating Incremental Loads (COPY INTO / Auto Loader)

■ Incremental Record Count Check

```
# After COPY INTO or Auto Loader, validate record counts
SELECT COUNT(*) as new_records FROM bronze.raw_events
WHERE _ingest_date = CURRENT_DATE();

-- Compare to yesterday's count
-- Alert if count is 0 or anomalously high/low
```

Q.2 Null Detection in Parsed JSON

■ Null Detection

```
# After from_json, check for unexpected nulls
SELECT
  SUM(CASE WHEN user_id IS NULL THEN 1 ELSE 0 END) as null_user_ids,
  SUM(CASE WHEN score IS NULL THEN 1 ELSE 0 END) as null_scores
FROM parsed_events
WHERE _ingest_ts >= CURRENT_TIMESTAMP() - INTERVAL 1 hour;

-- Raise alert if nulls > threshold
```

Q.3 Duplicate Detection (COPY INTO Force Mode)

■ Duplicate Detection

```
-- If re-running COPY INTO with force=true, check for duplicates
SELECT customer_id, COUNT(*) as cnt
FROM bronze.customers
WHERE _ingest_date >= CURRENT_DATE()
GROUP BY customer_id
HAVING cnt > 1;

-- If duplicates found, use MERGE to deduplicate in silver layer
```

MATRIX **Connector Comparison Matrix** All features at a glance

Complete Feature Comparison

Feature	Lakeflow SaaS	Database CDC	Query-Based	JDBC	Auto Loader	COPY INTO
Effort	UI-only	Moderate	Moderate	High (code)	High (code)	Moderate
Setup time	Minutes	Hours	Hours	Days	Days	Hours
Latency	Hourly+	Sub-minute	Batch	Batch	Streaming	Batch
Gateway	No	Yes	No	No	No	No
CDC support	App-dependent	Yes	No	No	No	No
Incremental	Watermark	CDC events	Cursor column	WHERE clause	Checkpointing	File tracking
Parallelization	Limited	Gateway	Database	4-option config	Partitions	N/A
Cost	Connector $ + compute	Gateway + compute	Compute	Compute	Compute	Compute
Best for	SaaS (Salesforce, etc.)	Real-time DB	Batch DB	Custom driver	File streaming	Cloud storage batch

When to Combine Connectors

Medallion Architecture Best Practice:

• **Bronze (raw):** Land data from multiple sources (Lakeflow + JDBC + REST)

• **Silver (cleaned):** Deduplicate, type-cast, handle nulls (single source of truth)

• **Gold (analytics-ready):** Aggregates, business logic, materialized views

PRAC2 **Practice Questions Round 2** 15 more questions

1. A Lakeflow SaaS connector syncs Salesforce hourly. Admin wants 15-min sync. Where to change?

A) Edit DLT pipeline JSON
B) Lakeflow Connect UI → edit schedule
C) Create new connector
D) CLI command

2. COPY INTO loads files; files arrive again next week. Re-ingest same files?

A) .mode('overwrite')
B) .option('force','true')
C) TRUNCATE + re-run
D) Manual dedup

3. Explode 1M rows × 10-item arrays. Estimated result?

A) 1M rows
B) ~10M rows
C) 100K rows
D) Undefined

4. REST API response: {"data":[...], "next_page": "URL"}. Best pagination?

A) Loop on next_page field
B) Counter-based (page=1,2,3)
C) Offset/limit
D) Cursor-based

5. JDBC fails: 'Connection timeout after 30sec'. Solution?

A) Increase numPartitions
B) Check network/DB availability
C) Use Auto Loader instead
D) Higher memory

6. Query-Based Connector syncs Oracle daily. Cursor column is 'MODIFIED_DATE'. If no changes today?

A) Empty ingestion
B) Loads previous day
C) Error
D) Skips run

7. from_json(raw, schema) where raw contains unknown fields not in schema?

A) Error thrown
B) Unknown fields dropped
C) New columns added (schema evolution)
D) Null

8. Workspace secret scope + Premium plan. Can you grant READ to data science group?

A) Yes, via databricks secrets put-acl
B) No, not supported
C) Yes, but only users
D) Through Unity Catalog

9. Lakeflow SaaS connector stalls. Check where first?

A) Connector logs in Spark UI
B) Lakeflow Jobs run history → last run status
C) Cluster startup logs
D) Unity Catalog audit

10. Auto Loader with 'file notification' vs 'directory listing'?

A) Notification = event-driven, scales
B) Listing = polls all files
C) Both identical
D) Listing faster

11. CDC gateway continues running between syncs, consuming compute?

A) Yes, always
B) No, only during sync window
C) Depends on config
D) User must manually stop

12. Merge two Lakeflow SaaS connectors' data in bronze (Salesforce + HubSpot)?

A) UNION ALL in silver layer
B) Can't merge different SaaS sources
C) Use MERGE INTO with keys
D) Auto-handled by Lakeflow

13. Query-Based Connector cursor column is auto-incremented ID (gaps exist). Reliable?

A) Yes, gaps don't matter
B) No, gaps cause data loss
C) Use timestamp instead
D) Must use MODIFIED_DATE

14. REST API rate-limited (100 req/min). Paginating 10K pages safely?

A) Add sleep() between requests
B) Databricks auto-handles
C) Use Lakeflow throttling
D) Fail & retry

15. JDBC read succeeds; output DataFrame has incorrect schema (string instead of bigint for IDs)?

A) Driver type inference wrong; cast explicitly
B) JDBC bug
C) Database schema issue
D) Spark bug

FLASH **Flashcard Summary** Memorization quick-hits

Q: Lakeflow SaaS Connectors

A: Salesforce, Workday, HubSpot, Jira, Zendesk, Google Analytics, Stripe — all UI-configured, no-code.

Q: Database Connectors (CDC)

A: MySQL, PostgreSQL, SQL Server — Change Data Capture; requires ingestion gateway; real-time.

Q: Query-Based Connectors (NEW)

A: Oracle, Teradata, MySQL, PostgreSQL — direct query without CDC; no gateway; GA May 2026.

Q: File Source Connectors

A: Google Drive, SharePoint — unstructured/structured file ingestion.

Q: Streaming Connectors

A: RabbitMQ, Kafka — continuous message consumption into streaming tables.

Q: JDBC Parallelism

A: 4 options required together: partitionColumn + lowerBound + upperBound + numPartitions.

Q: Workspace Secret Scope

A: scope.key namespace; workspace-local; dbutils.secrets.get('scope', 'key').

Q: UC Secret

A: catalog.schema.secret namespace; metastore-level; UC privileges; audit trail.

Q: Colon Operator

A: raw:user.id (Databricks SQL only); extracts from JSON STRING.

Q: from_json

A: Parses JSON STRING to StructType; requires schema.

Q: get_json_object

A: Extracts single field from JSON STRING as STRING; JSONPath syntax.

Q: explode()

A: Converts ArrayType to rows; one row per element.

Q: Decision Framework

A: SaaS→Lakeflow. DB batch→Query-based. DB real-time→CDC. Files→COPY/Auto Loader. Custom→REST/JDBC.

Q: Incremental Sync

A: Tracks changes via watermark (SaaS), CDC events (DB CDC), cursor column (Query-based), or file tracking (COPY/Auto Loader).

Q: Lakeflow Jobs

A: Auto-created by managed connectors; orchestrates sync schedules; appears in Lakeflow Jobs UI.

FINAL Final Exam Readiness Checklist 100% coverage

■ Domain 1 (6%): Lakehouse architecture, Delta Lake ACID, Unity Catalog basics, compute types

■ Domain 2 (21%) — **YOU ARE HERE**:

■ Connector types: 6 types, benefits, when to use each

■ SaaS connectors: Salesforce, Workday, HubSpot, etc.

■ Database CDC: ingestion gateway, real-time, change events

■ Query-Based: cursor columns, no gateway, May 2026 GA

■ JDBC: parallel reads (ALL 4 options), driver/URL format

■ REST API: pagination, requests library, spark.createDataFrame

■ Semi-structured: JSON functions (from_json, get_json_object, colon op, explode)

■ Secrets: workspace scopes vs UC secrets (namespace, governance)

■ Auto Loader: checkpointing, schema evolution, directory listing vs file notification

■ COPY INTO: idempotent, file tracking, force=true for re-ingestion

■ Decision framework: all scenarios covered

■ Incremental sync methods: watermark, CDC, cursor, file tracking

■ Domain 3 (22%): Bronze→Silver→Gold, joins, aggregations, materialized views [**NEXT**]

■ Domain 4 (16%): Lakeflow Jobs, DAG, retries, triggers [AFTER PHASE 3]

■ Domain 5 (10%): CI/CD, Git Folders, Declarative Bundles [AFTER PHASE 3]

■ Domain 6 (10%): Troubleshooting, Spark UI, skew, OOM, Liquid Clustering [WEEK 11]

■ Domain 7 (15%): Unity Catalog security, GRANT/REVOKE, masking, ABAC [WEEK 11]

■ **Domain 2 Complete — 21% of exam locked in!**

■ **Cumulative: 27% (Domain 1 + 2)**

■ **Momentum: Phase 2 done → Phase 3 (Transformation) is next**

REVIEW **Phase 2 Review** Weeks 3-5 summary

Week 3 Recap — COPY INTO

- COPY INTO for batch cloud storage (S3, ADLS, GCS)
- Idempotent: tracks loaded files, skips duplicates
- force=true to re-ingest already-loaded files
- VALIDATE mode for dry-run
- Simple SQL, no custom code

Week 4 Recap — Auto Loader

- cloudFiles for continuous file ingestion
- Streaming or batch mode
- Checkpointing for exactly-once semantics
- Schema inference & evolution
- Directory listing vs file notification (event-driven)
- Rescues bad data via _rescued_data column

Week 5 Recap — Lakeflow Connect + JDBC + REST + JSON

- 6 connector types: SaaS, CDC, Query-based, File, Streaming, Community
- Query-Based NEW (May 2026): no CDC, simpler setup
- JDBC: parallel reads with 4-option requirement
- REST: pagination, requests library
- JSON: from_json (StructType), get_json_object (STRING), colon op, explode
- Secrets: workspace scopes vs UC secrets (governance & namespace)

Phase 2 Totals: Domain 2 = 21% of Exam (~9 Questions)

Week	Focus Area	Key Concepts	Delivery Method
3	COPY INTO	Idempotent, file tracking, SQL	Batch from cloud
4	Auto Loader	Streaming, checkpointing, schema evolution	Files streaming/batch
5	Lakeflow + JDBC + REST	6 connector types, parallelization, JSON parsing	SaaS, DB, API, JSON

CMD **Command & API Reference** Quick copy-paste

Databricks CLI — Secrets & Connections

Command	Purpose
databricks secrets create-scope NAME	Create workspace secret scope
databricks secrets put-secret SCOPE KEY	Add secret to scope (prompts for value)
databricks secrets get-acl SCOPE	List ACLs on scope
databricks secrets put-acl SCOPE PRINCIPAL READ	Grant READ permission
databricks connections create --type TYPE --name NAME	Create UC connection
databricks cli version	Check CLI version

Python — Common Patterns

Read secrets: user = dbutils.secrets.get('scope', 'key')
Create DataFrame from list: df = spark.createDataFrame(all_records)
Parse JSON: df.withColumn('parsed', from_json(col('raw'), schema))
Flatten arrays: df.select(col('id'), explode(col('items')).alias('item'))
Extract single JSON field: get_json_object(col('raw'), '$.user.id')
Colon operator (SQL only): SELECT raw:user.id FROM table

SQL — Common Patterns

Pattern	SQL Syntax
Extract JSON string	SELECT raw:user.id, raw:events[0].type FROM table
Count records loaded today	SELECT COUNT(*) FROM bronze.raw WHERE _ingest_date = CURRENT_DATE()
Find duplicates	SELECT id, COUNT(*) as cnt FROM table GROUP BY id HAVING cnt > 1
Check for nulls	SELECT COUNT(*) as null_count FROM table WHERE key_column IS NULL
Validate schema evolution	DESCRIBE TABLE schema_name

ANS **Mock Exam Answers** Detailed explanations

Q1: Answer B

Query-Based Connectors (May 2026 GA) query the source directly without requiring CDC configuration or an ingestion gateway. Simpler setup for periodic batch workloads.

Q2: Answer B

Event-driven (file notification) doesn't require scanning the entire directory every sync. Directory listing polls all files, causing higher compute. Notification scales better for billions of files.

Q3: Answer A

.option('force','true') overrides COPY INTO's idempotency check. This forces re-ingestion of already-loaded files.

Q4: Answer B

spark.createDataFrame(list) converts a Python list of dicts to a Spark DataFrame. Then write to Delta with .write.format('delta').

Q5: Answer B

Lakeflow Connect UI exposes scheduling. You don't edit the underlying DLT pipeline—UI handles orchestration. Databricks auto-creates the Lakeflow Job with new schedule.

DONE	You've Completed Phase 2!	Domain 2 Mastery Achieved

Achievement Unlocked

You have now mastered all ingestion methods and tools for Databricks. Weeks 1-5 = **27% of exam** (Domain 1 Platform + Domain 2 Ingestion).

Skills Demonstrated

- ✓ Understand Databricks Lakehouse architecture & Delta Lake fundamentals
- ✓ Master all 6 Lakeflow Connect connector types (NEW: Query-based GA May 2026)
- ✓ Design idempotent batch ingestion (COPY INTO) with file tracking
- ✓ Configure streaming ingestion (Auto Loader) with schema evolution
- ✓ Parallelize JDBC reads from relational databases
- ✓ Ingest from custom REST APIs with pagination
- ✓ Parse and flatten semi-structured JSON data
- ✓ Manage secrets (workspace scopes + Unity Catalog secrets)
- ✓ Apply the Domain 2 decision framework to any ingestion scenario
- ✓ Troubleshoot common ingestion issues (connectivity, schema, checkpointing)

Next Steps: Phase 3 (Weeks 6-8)

Domain 3 is 22% of exam and focuses on transforming ingested data into analytics-ready formats. Topics include Bronze-Silver-Gold patterns, all join types, aggregations, materialized views, and data quality checks.

KEY **Key Takeaways** Week 5 in 2 pages

1. Connector Landscape Simplified (Official Terminology)

OLD (incorrect): Standard connectors vs Managed connectors

NEW (2026): 6 connector types: SaaS, Database (CDC), Query-based, File source, Streaming, Community

2. Query-Based Connectors Are Game-Changing (May 2026 GA)

For periodic batch loads from databases (Oracle, Teradata, MySQL, PostgreSQL, SQL Server), Query-Based connectors eliminate CDC complexity. Use cursor columns; no gateway required.

3. JDBC Parallelization = 4 Options or Nothing

Setting only partitionColumn does nothing. Must set ALL 4 together: partitionColumn + lowerBound + upperBound + numPartitions.

4. Secrets: Two Paths Forward

Workspace scopes (dbutils.secrets.get) for prototyping. UC secrets (catalog.schema.secret) for production (multi-workspace, full governance, audit).

5. JSON Functions: Know the Difference

from_json(col, schema): Parses JSON STRING into typed StructType. Requires schema.

get_json_object(col, path): Extracts one field as STRING. JSONPath syntax.

Colon operator (raw:field): SQL-only. Shorthand for JSON extraction.

explode(array_col): Converts ArrayType to rows.

6. Decision Framework Locked In

SaaS source → Lakeflow SaaS.

Relational DB, periodic batch → Query-Based Connector.

Relational DB, real-time → Database (CDC).

Cloud storage batch → COPY INTO.

Files streaming → Auto Loader.

Custom API → REST.

Custom JDBC → JDBC.

Nested JSON/arrays → JSON functions.

7. All Connectors Land in Unity Catalog

Every ingestion method writes to streaming tables or Delta tables in Unity Catalog. Governance (GRANT/REVOKE), audit trails, time-travel—all built-in via UC.

PLAN **Study Plan Forward** Weeks 6-12 roadmap

Current Status: 27% of Exam Secured

✓ Domain 1 (6%): Lakehouse, Delta, UC basics

✓ Domain 2 (21%): Ingestion (COPY, Auto Loader, Lakeflow, JDBC, REST, JSON)

Phase 3 — Weeks 6-8: Data Transformation (Domain 3, 22%)

Focus: Bronze-Silver-Gold medallion architecture, joins (inner/left/broadcast), aggregations, materialized views, streaming tables, data quality constraints.

Phase 4 — Weeks 9-10: Orchestration & CI/CD (Domains 4-5, 26%)

Focus: Lakeflow Jobs, triggers, retries, error handling. CI/CD via Declarative Bundles, Git Folders, Databricks CLI, source control best practices.

Phase 5 — Weeks 11-12: Advanced Topics & Final Review (Domains 6-7, 25%)

Focus: Troubleshooting (Spark UI, skew, OOM, Liquid Clustering). Governance & Security (Unity Catalog, GRANT/REVOKE/DENY, masking, ABAC, audit logs).

Milestone Timeline

→ End of Week 5: 27% (Domains 1-2 ✓)

→ End of Week 8: 49% (Domains 1-3 ✓)

→ End of Week 10: 75% (Domains 1-5 ✓)

→ End of Week 12: 100% (All domains ✓) — READY TO TEST

You are 1/4 of the way through the full 12-week program. Momentum is building. Keep going!